tamamdır şimdi senden istediğim şey ise bütün bu proje hakkında konuştuklarımızı birleştirip bir "workthrough" tarzında bir plan yazman. projenin fikri, içeriği, nasıl bir hizmet sunduğu, amacı, teorikte nasıl yapıldığı hangi özellikleri barındırdığı, teknik açıdan nasıl bir temele sahip olacağı, web sitesinin tasarımının nasıl yapılacağı ve hangi özelliklere sahip olacağı vs vs vsvs

ANLAYACAĞIN PROJE TAMAMDIR ŞUAN KODLAMA KISMINI BAŞKA AI İLE YAPICAM DOLAYISIYLA PROJEYİ ONA ANLATACAĞIM

Aşağıdaki dokümanı doğrudan kod yazan AI'a verebilirsin. Amaç, projeyi **ürün vizyonu, veri modeli, backend pipeline, frontend ekranları ve tasarım sistemi** olarak tek bir "build brief" hâline getirmek.

SignalCraft — Build Brief & Workthrough

1. Proje Özeti

Ürün adı: SignalCraft
Tek cümlelik tanım: Teknoloji ve AI içerik üreticileri için, yükselen konuları keşfeden, kişiselleştirilmiş içerik fırsatlarını radarlayan ve bu sinyalleri yayınlanabilir içerik brief'lerine dönüştüren web uygulaması.

Ana problem: Teknoloji/AI içerik üreticileri; haber takibi, trend doğrulama, içerik fikri bulma, çoklu platforma uyarlama ve içerik üretimi için ayrı ayrı araçlar kullanıyor. Bu süreç dağınık, yavaş ve tekrar eden bir iş yükü yaratıyor.

Çözüm: SignalCraft üç modülü tek akışta birleştirir:

1. **Discover:** Teknoloji/AI dünyasındaki güncel olayları kategorize edilmiş bir haber/keşif akışı olarak sunar.
2. **Radar:** Kullanıcının nişine göre sinyalleri filtreler, skorlar ve içerik fırsatlarına dönüştürür.
3. **Studio:** Seçilen story'yi platforma özel, düzenlenebilir içerik brief'ine dönüştürür.

Kullanıcı yolculuğu:

```
Discover → Radar → Studio
```

- Kullanıcı önce genel teknoloji haberlerini keşfeder.
- Sonra kendi nişine göre öne çıkan fırsatları görür.
- En sonunda seçtiği konudan X thread, LinkedIn postu, YouTube Shorts hook'u, newsletter bölümü veya blog taslağı üretir.

2. Hedef Kullanıcı

Birincil hedef kitle

- AI, yazılım geliştirme, startup, ürün geliştirme ve developer tooling hakkında içerik üreten kişiler.
- 1K-100K takipçili X, LinkedIn, YouTube veya newsletter üreticileri.
- Indie hackerlar, developer advocate'ler, teknoloji meraklıları.

İkincil hedef kitle

- Küçük ajanslarda creator içerik stratejisi hazırlayan kişiler.
- B2B SaaS şirketlerinde content/social medya çalışanları.
- Teknoloji alanında bülten çıkaran kişiler.

Kullanıcının ana vaadi

> "Her sabah 15 dakikada, paylaşmaya değer 5 konu ve her biri için hazır içerik brief'i al."

3. Ürün Modülleri

3.1 Discover

Discover, ürünün herkese açık ya da kayıt sonrası erişilebilen **haber/keşif akışıdır**. X'in keşfet akışı ile teknoloji haber sitesinin ortası gibi düşünülmelidir.

Amaç: Kullanıcıya "teknoloji dünyasında şu an ne oluyor?" sorusunun cevabını vermek.

Özellikler:

- Kategorilere ayrılmış story akışı.
- Her story, aynı olaya dair birden fazla kaynağın kümelendiği tek bir kart.
- Kategori filtreleri: AI Models, AI Tools, Developer Tools, Open Source, Startups, Security, Web & Frontend, Mobile, Data & Infrastructure.
- Zaman filtresi: Today, Last 24 hours, This week.
- Arama.
- Trending now paneli.

- Yeni AI araçları paneli.
- GitHub'da yükselen projeler paneli.
- Story detail sayfası.
- "Radar'a ekle" ve "Studio'da üret" aksiyonları.

Story kartı içeriği:

- Kategori etiketi.
- Başlık.
- 2-3 cümlelik özet.
- Kaynak sayısı.
- İlk görülme zamanı.
- Velocity skoru.
- HN puanı, Reddit upvote/yorum sayısı, GitHub yıldız/release bilgisi gibi platform sinyalleri.
- Kaynak linkleri.
- "Neden önemli?" açıklaması.
- "Radar'a ekle" butonu.
- "Studio'da üret" butonu.

3.2 Radar

Radar, kullanıcının tanımladığı nişe göre kişiselleştirilmiş sinyal panosudur.

Amaç: "Benim nişim için bugün hangi içerik fırsatları var?" sorusunu yanıtlamak.

Kullanıcı kurulum alanları:

- İlgi alanları / nişler: örneğin "AI agents", "developer tools", "indie hacking".
- Hedef platformlar: X, LinkedIn, YouTube, Newsletter.
- İçerik dili: Türkçe, İngilizce.
- Ton: teknik, analitik, eğlenceli, karşıt görüşlü, sade.
- Takip edilen hesaplar veya kanallar.
- Negatif anahtar kelimeler: "bu konuları gösterme".

Radar çıktıları:

- **High signal:** Yükselişi en yüksek story'ler.
- **Early signal:** Henüz az konuşulan ama hızla yükselen konular.
- **Content gap:** İlgi çekici ancak iyi kapsanmamış içerik açıları.
- **Personal angle:** Kullanıcının nişine uygun içerik açıları.
- **Daily digest:** Günlük e-posta özeti.

- **Alert:** Belirlenen eşiği aşan story için bildirim.

Skorlama formülü:

```
Score =
  0.35 × Velocity
+ 0.25 × Engagement
+ 0.20 × NicheFit
+ 0.10 × SourceDiversity
+ 0.10 × Freshness
```

Amaç "en viral" içeriği bulmak değil, **içerik üretmeye değer** sinyali bulmaktır.

3.3 Studio

Studio, seçilen story'yi kullanıcının marka sesine ve hedef platformuna göre içerik üretimine dönüştürür.

Girdiler:

- Story ID.
- Hedef platform: X, LinkedIn, YouTube Shorts, Reels, Newsletter, Blog.
- Format: thread, tek post, video hook, video script, newsletter bölümü, blog taslağı.
- Ton.
- Hedef kitle.
- Uzunluk.
- Kullanıcının kendi görüşü veya ek bağlamı.
- Marka sesi örnekleri.

Çıktılar:

- 10 hook önerisi.
- 3 başlık alternatifi.
- 1 X thread taslağı.
- 1 LinkedIn postu.
- 1 YouTube Shorts hook seti.
- 1 Shorts video scripti.
- 1 newsletter paragrafı.
- 1 blog taslağı.
- Kaynak linkleri ve atıflar.
- AI kullanım beyanı.

Önemli: Studio çıktıları düzenlenebilir olmalı. Kullanıcı metni düzenleyebilir, farklı hook seçebilir, yeniden üretebilir ve brief'i kaydedebilir.

4. Ürün Felsefesi

SignalCraft bir “AI içerik üreticisi” değil, **içerik üreticisinin günlük komuta merkezi**dir.

Ürünün vaadi şu üç cümlede özetlenir:

1. **Discover:** “Bugün teknoloji dünyasında ne oldu?”
2. **Radar:** “Benim nişim için hangisi içerik fırsatı?”
3. **Studio:** “Bunu nasıl içerik yaparım?”

Ürün, kullanıcının yerine içerik üretmez. Kullanıcıya güçlü bir başlangıç noktası, kaynaklar ve düzenlenebilir taslak sunar.

5. Maliyet ve Ölçek Stratejisi

İlk sürüm mümkün olduğunca ücretsiz çalışmalıdır.

Temel prensipler

- Her ham içerik için LLM çağrısı yapılmaz.
- Önce ucuz filtreleme, dedupe ve skorlama yapılır.
- LLM yalnızca seçilmiş, kümelendirilmiş ve yüksek potansiyelli story’ler için çalışır.
- Studio üretimi kullanıcı tetiklemelidir.
- X gibi pahalı API’ler ilk sürümde kullanılmaz ya da çok sınırlı kullanılır.
- YouTube API arama yerine kanal RSS feed’leri ile kullanılır.

Önerilen ücretsiz/ucuz altyapı

Katman	Tercih
Web + API	Next.js on Vercel Hobby
Database + Auth	Supabase Free + Postgres + pgvector
Background jobs	Inngest Free
Email	Resend Free
LLM	Gemini Flash / GPT mini / Groq ücretsiz kotaları
Embedding	Ücretsiz embedding kotası veya basit dedupe ile başlangıç
Analytics	PostHog free tier
Error tracking	Sentry free tier

Pipeline maliyet disiplini

```
Fetch
→ Normalize
→ Dedupe
→ Rule-based filtering
→ Score
→ Select top N
→ LLM summary
→ Discover
```

Günde 1.000 ham item gelse bile LLM yalnızca 20-40 story için çalışmalıdır.

6. Veri Kaynakları

MVP kaynak seti

İlk sürümde 20-40 kaynak yeterlidir.

RSS kaynakları

- OpenAI Blog
- Anthropic News
- Google DeepMind Blog
- Hugging Face Blog
- GitHub Blog
- Vercel Blog
- TechCrunch AI
- The Verge AI
- Ars Technica AI
- VentureBeat AI
- Ürün günlükleri: Supabase, Stripe, Cloudflare, Linear vb.

Topluluk kaynakları

- Hacker News front page.
- Hacker News “Show HN”.
- Reddit: r/MachineLearning, r/LocalLLaMA, r/webdev, r/SideProject, r/singularity.
- Product Hunt.
- GitHub Trending / release feed’leri.
- arXiv: cs.AI, cs.CL, cs.LG.

YouTube

- 30-50 seçilmiş teknoloji/AI kanalının RSS feed'i.
- YouTube Data API yalnızca doğrulama ve detay için, sınırlı kullanılmalı.

X

İlk sürümde X kullanılmayabilir. Daha sonra yalnızca:

- Seçilmiş 100-300 creator/researcher hesabı.
- Sınırlı sayıda günlük sorgu.
- Yalnızca yüksek skorlu story'ler için amplifikasyon sinyali.

7. Veri Pipeline

Pipeline akışı

```
Source Registry
→ Fetcher
→ Normalizer
→ Deduplicator
→ Classifier / Filter
→ Embedder
→ Clusterer
→ Story Builder
→ Ranker
→ Discover / Radar / Studio
```

7.1 Fetcher

Her kaynak için ayrı fetcher yazılmalı.

- RSS fetcher.
- HN Algolia API fetcher.
- GitHub API fetcher.
- Reddit API fetcher.
- Product Hunt fetcher.
- arXiv fetcher.
- YouTube RSS fetcher.

Frekans önerisi:

- RSS: 15-30 dakika.
- HN: 15-60 dakika.
- GitHub: 30-60 dakika.

- Reddit: 30-60 dakika.
- YouTube RSS: 1-6 saat.
- X: MVP'de yok.

7.2 Normalizer

Her ham item şu alanlara normalize edilir:

```
{
  source_id,
  external_id,
  url,
  canonical_url,
  title,
  description,
  content_text,
  author,
  published_at,
  fetched_at,
  language,
  source_type,
  raw_metadata
}
```

7.3 Dedupe

Üç aşamalı dedupe yapılmalı:

1. **Exact match:** Canonical URL + content hash.
2. **Near match:** Başlık/body benzerliği, SimHash veya shingle yöntemi.
3. **Semantic match:** Embedding + pgvector; örneğin cosine similarity 0.85 üzeri ise aynı cluster.

Amaç aynı olayın 10 farklı kart olarak görünmesini engellemektir.

7.4 Classifier / Filter

LLM kullanmadan önce kural tabanlı filtre uygulanır:

- Dil kontrolü.
- Spam kontrolü.
- Niş dışı içerik kontrolü.
- Duplicate kontrolü.
- Kaynak güvenilirliği.
- Yayın tarihi.
- Minimum içerik uzunluğu.

- Yasaklı kelimeler.

7.5 Story Builder

Sadece filtrelenmiş ve kümelendirilmiş içerikler için LLM çalışır.

LLM çıktısı:

```
{
  title,
  summary,
  why_it_matters,
  category,
  entities,
  sentiment,
  content_creator_relevance,
  suggested_angles
}
```

7.6 Ranker

Her story şu metriklerle skorlanır:

- Velocity: Son 24 saatteki yükseliş hızı.
- Engagement: HN puanı, Reddit etkileşimi, GitHub yıldızı vb.
- NicheFit: Kullanıcının nişleriyle eşleşme.
- SourceDiversity: Kaç farklı kaynaktan destekleniyor.
- Freshness: Ne kadar yeni.
- Trust: Kaynak güvenilirliği.

8. Veritabanı Modeli

Temel tablolar

```
users
- id
- email
- name
- plan
- created_at

niches
- id
- user_id
- name
- keywords
- excluded_keywords
- platforms
- tone
```

```
- language
- created_at

sources
- id
- name
- type // rss, hn, reddit, github, youtube, producthunt
- url
- trust_weight
- active
- created_at

raw_items
- id
- source_id
- external_id
- url
- canonical_url
- title
- description
- content_text
- author
- published_at
- fetched_at
- language
- content_hash
- raw_metadata

stories
- id
- title
- summary
- why_it_matters
- category
- entities
- velocity_score
- engagement_score
- niche_fit_score
- source_diversity_score
- freshness_score
- final_score
- first_seen_at
- last_updated_at
- status

story_sources
- id
- story_id
- raw_item_id
- source_id

story_clusters
- id
- story_id
- embedding
- similarity_threshold
```

```
- created_at

trend_scores
- id
- story_id
- metric
- value
- calculated_at

user_signals
- id
- user_id
- story_id
- action // save, hide, boost, generate
- created_at

briefs
- id
- user_id
- story_id
- platform
- format
- tone
- output
- status
- created_at

subscriptions
- id
- user_id
- plan
- status
- current_period_end
```

9. Teknik Mimari

Önerilen stack

Katman	Teknoloji
Framework	Next.js App Router
UI	Tailwind CSS + shadcn/ui
Database	Supabase Postgres
Auth	Supabase Auth
Vector search	pgvector
Background jobs	Inngest
LLM	Claude / GPT / Gemini, model göre görev bazlı seçilir
Email	Resend

Katman	Teknoloji
Payments	Stripe
Analytics	PostHog
Error tracking	Sentry
Hosting	Vercel
Icons	Lucide
Charts	Recharts veya hafif SVG sparkline

Mimari prensipler

- Uzun süren LLM işleri HTTP request içinde çalışmaz; background job'a alınır.
- Fetcher, normalizer, dedupe, embedding ve LLM işlemleri ayrı job adımlarıdır.
- Her job idempotent olmalı; aynı item tekrar işlenmemeli.
- LLM çağrıları için retry, timeout ve maliyet limiti olmalı.
- Kullanıcı bazlı LLM üretimi rate limit ile korunmalı.
- Free plan kullanıcıları için günlük brief ve radar limiti olmalı.

10. API / Route Yapısı

Public routes

```
GET  /api/discover
GET  /api/discover/[storyId]
GET  /api/categories
GET  /api/trending
```

Auth routes

```
GET  /api/radar
POST /api/radar/niches
PUT  /api/radar/niches/[id]
POST /api/radar/[storyId]/save
POST /api/radar/[storyId]/hide

POST /api/studio/generate
GET  /api/studio/briefs
GET  /api/studio/briefs/[id]
PUT  /api/studio/briefs/[id]
```

Internal jobs

```
POST /jobs/fetch-sources
POST /jobs/normalize-items
POST /jobs/dedupe-items
POST /jobs/build-stories
POST /jobs/calculate-scores
POST /jobs/send-daily-digest
```

11. Web Sitesi Tasarımı

Tasarım konsepti

“Signal Terminal”

SignalCraft; Linear benzeri koyu, hassas, teknik bir SaaS estetiği ile haber terminali hissini birleştirir. Amaç “generic AI product” görünümü değil; **yüksek sinyalli, hızlı taranabilir, geliştirici dostu bir radar** hissidir.

Tasarım ilkeleri

- Dark-first tasarım.
- Saf siyah yerine hafif soğuk koyu zemin.
- Tek belirgin accent rengi.
- Az gradyan, az parlama.
- Bilgi yoğunluğu yüksek ama okunabilir arayüz.
- Monospace detaylar: skor, zaman, kaynak sayısı.
- Hareket küçük, hızlı ve işlevsel.
- Landing page’de gerçek ürün ekranı gösterilmeli.

Renk sistemi

```
--color-canvas: #0A0B0D;
--color-surface-1: #111318;
--color-surface-2: #171A21;
--color-border: rgba(255, 255, 255, 0.08);
--color-text-primary: #F5F7FA;
--color-text-secondary: #9AA3B2;
--color-accent: #00E5A0;
--color-warning: #FFB224;
--color-danger: #FF5C5C;
```

Accent rengi yalnızca şu durumlarda kullanılmalı:

- Primary CTA.
- Aktif sinyal.

- Yeni story.
- Yüksek skor.
- Başarılı işlem.

Mor-siyah AI gradyanları kullanılmamalı.

Tipografi

- **UI/Display:** Geist, Inter Variable veya General Sans.
- **Mono:** JetBrains Mono veya Geist Mono.

```
Hero title: 56-72px, weight 600-650, letter-spacing -0.03em
Section title: 32-40px
Story title: 18-20px
Body: 15-16px, line-height 1.55
Meta: 12-13px mono
```

Landing page bölümleri

1. **Hero**

```
SignalCraft

Teknoloji içerik üreticileri için günlük sinyal radarı.

Yükselen AI ve geliştirici konularını keşfet,
nişine göre fırsatları gör,
dakikalar içinde yayınlanabilir içerik üret.

[Ücretsiz başla] [Canlı Discover'ı gör]
```

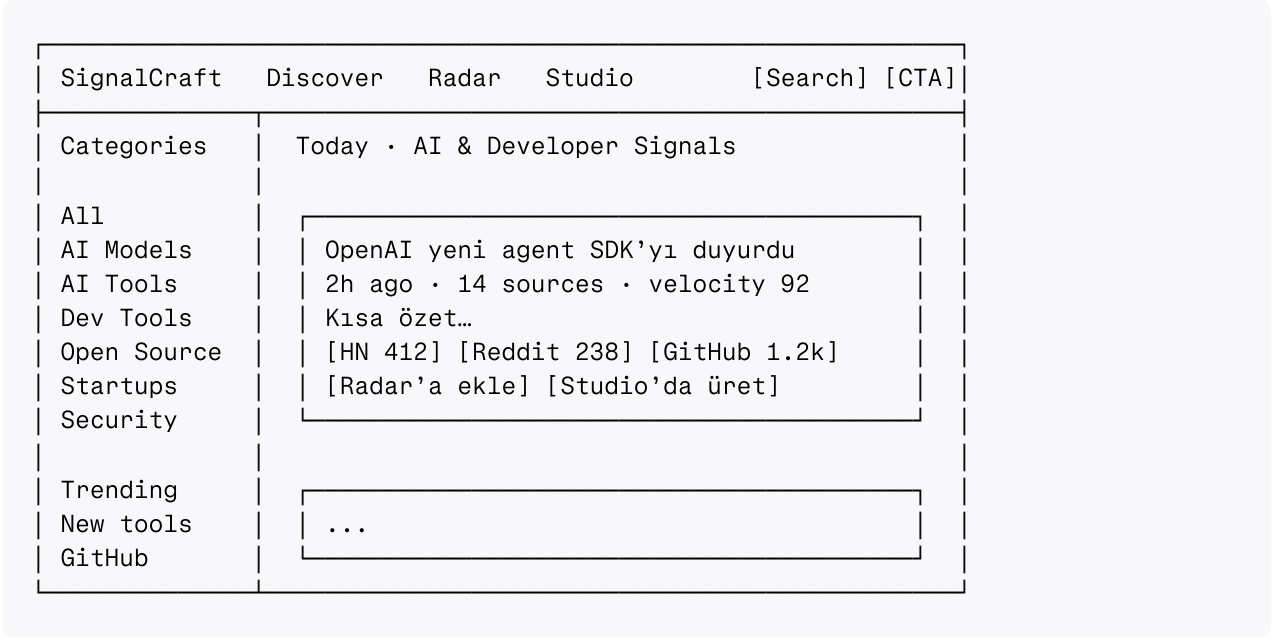
2. **Gerçek ürün mockup'ı**
   Hero altında gerçek Discover ekranı gösterilmeli.
3. **Üç modül bölümü**
   Bento grid ile Discover, Radar ve Studio tanıtılmalı.
4. **Problem bölümü**
   "Fikir bulmak, trendi doğrulamak ve her platforma uyarlama ayrı ayrı zaman alıyor."
5. **Nasıl çalışır**
   5 adımlı pipeline görselleştirilmeli.
6. **Örnek çıktı**
   Gerçek bir story'den üretilmiş X thread, LinkedIn postu ve Shorts hook'u gösterilmeli.
7. **Fiyatlandırma**
   Free / Creator / Pro.
8. **SSS**
   Kaynak doğruluğu, telif, AI kullanımı, veri güncelliği.

Landing page görsel dili

- Koyu zemin.
- İnce 1px border’lar.
- Hafif surface katmanları.
- Cyan-green accent.
- Monospace teknik etiketler.
- Gerçek UI ekranları.
- Bento grid.
- Hafif hover glow.
- Aşırı 3D, WebGL veya video duvarı yok.

12. Discover UI Tasarımı

Layout



Story card

- Sol kenarda ince “signal bar”.
- Kategori etiketi.
- Başlık.
- Özet.
- Velocity skoru.
- Kaynak sayısı.
- Zaman.
- HN/Reddit/GitHub sinyalleri.

- “Radar’a ekle”.
- “Studio’da üret”.

Signal bar renkleri:

- Yeşil: High signal.
- Sarı: Early signal.
- Gri: Normal.
- Kırmızı: Düşük güven.

13. Radar UI Tasarımı

Radar ekranı üç bölümden oluşmalı:

```
High signal
Early signal
Content gaps
```

Her satırda:

- Story başlığı.
- Skor.
- Yükseliş grafiği.
- Kaynak sayısı.
- “Neden önemli?”
- “Senin açın”.
- “Brief oluştur”.

Radar, kullanıcının niş ayarlarına göre farklı sıralama ve etiketler göstermelidir.

14. Studio UI Tasarımı

Studio sade ve odaklı olmalı.

```
┌──────────────────────────────────────────────┐
│ Story: OpenAI agent SDK duyurusu             │
│ Platform: X Thread | Ton: Analitik           │
├──────────────────────────────────────────────┤
│ Hook options                                 │
│ 1. ...                                       │
│ 2. ...                                       │
│ 3. ...                                       │
├──────────────────────────────────────────────┤
│ Thread draft                                 │
│ [editable textarea]                          │
├──────────────────────────────────────────────┤
│ Sources                                      │
│ - OpenAI blog                                │
```

```
| - HN discussion                                    |
└────────────────────────────────────────────────────┘
```

Sağ panelde:

- Platform seçimi.
- Format seçimi.
- Ton seçimi.
- Hedef kitle.
- Uzunluk.
- Kullanıcının kendi görüşü.
- “Yeniden üret”.
- “Kaydet”.

15. Fiyatlandırma

Plan	Fiyat	Kapsam
Free	$0	Günde 3 radar sinyali, temel Discover, 1 niş, sınırlı Studio üretimi
Creator	$19-29/ay	Sınırsız brief, 3 niş, günlük digest, kayıtlı fikir havuzu
Pro	$49-79/ay	Çoklu niş, marka sesi, içerik takvimi, API/webhook, ekip erişimi

İlk sürümde yalnızca Free ve Creator planı yeterli olabilir. Pro planı sonraya bırakılabilir.

16. MVP Kapsamı

MVP’de yapılacaklar

- Supabase auth.
- Kullanıcı niş kurulumu.
- RSS, HN, GitHub, Product Hunt, arXiv kaynakları.
- Fetcher worker.
- Normalizer.
- Dedupe.
- Basit embedding/cluster.
- Story oluşturma.
- Discover feed.
- Story detail.
- Kategori filtreleri.
- Arama.

- Radar skorlama.
- Günlük digest.
- Studio brief generator.
- Brief kaydetme.
- Stripe ile Creator planı.

MVP’de yapılmayacaklar

- Otomatik sosyal medya yayını.
- Video editing.
- AI video/görsel üretimi.
- X API entegrasyonu.
- Ekip iş birliği.
- Mobil uygulama.
- Gelişmiş recommendation sistemi.
- Çok dilli içerik üretimi.
- Tüm platform entegrasyonları.

17. Geliştirme Sırası

Hafta 1: Temel altyapı

- Next.js + Tailwind + shadcn/ui kurulumu.
- Supabase auth.
- Temel layout.
- Database schema.
- Source registry.
- RSS fetcher.

Hafta 2: Discover

- Normalizer.
- Dedupe.
- Embedding.
- Clustering.
- Story builder.
- Discover feed.
- Story detail.
- Kategori filtreleri.

Hafta 3: Radar

- Niş kurulumu.
- Skorlama.
- Kişiselleştirilmiş feed.
- Günlük digest.
- E-posta entegrasyonu.

Hafta 4: Studio

- Brief generator.
- Platform/format seçimi.
- LLM prompt'ları.
- Düzenlenebilir çıktı.
- Brief kaydetme.

Hafta 5: Monetizasyon

- Stripe.
- Free/Creator plan limitleri.
- Usage tracking.
- Billing portal.

Hafta 6: Beta

- 10-20 beta kullanıcı.
- Onboarding.
- Feedback collection.
- Retention ölçümü.
- Pipeline kalibrasyonu.

18. Başarı Metrikleri

Activation

- Kullanıcı niş kurulumunu tamamladı mı?
- İlk 7 gün içinde en az bir brief üretti mi?
- İlk brief'ten sonra içerik yayınladı mı?

Engagement

- Günlük Discover açma oranı.
- Radar’a eklenen story sayısı.
- Studio’da üretilen brief sayısı.
- Brief’ten sonra düzenleme oranı.

Retention

- 7. gün retention.
- 30. gün retention.
- Haftalık aktif kullanıcı.
- Günlük digest açılma oranı.

En kritik metrik

```
Bir brief’ten kaç içerik gerçekten yayınlandı?
```

Bu metrik, ürünün “güzel özet üreten AI” değil, içerik üretim sürecinin gerçek bir parçası olduğunu gösterir.

19. Kod Yazan AI İçin Net Talimatlar

1. Projeyi Next.js App Router ile kur.
2. Tailwind CSS ve shadcn/ui kullan.
3. Supabase’i auth, Postgres ve pgvector için kullan.
4. Background işler için Inngest kullan.
5. LLM çağrılarını background job’larda çalıştır.
6. Her ham item için LLM çağırma.
7. Önce dedupe, filtre ve skorlama yap; sonra LLM çalıştır.
8. Studio üretimini kullanıcı tetiklemeli yap.
9. Discover’ı herkese açık ya da kayıt sonrası erişilebilir yap.
10. Radar’ı yalnızca giriş yapmış kullanıcıya aç.
11. Studio çıktılarını editable brief olarak sakla.
12. Tasarımı “Signal Terminal” konseptine göre kur.
13. Mor-siyah generic AI estetiği kullanma.
14. Tek accent rengi olarak cyan-green kullan.
15. Monospace fontu yalnızca skor, zaman ve teknik metadata için kullan.
16. Landing page’de gerçek Discover ekranını göster.

17. İlk sürümde X, video editing ve otomatik yayınlama ekleme.
18. MVP'de ücretsiz altyapı limitlerine uygun, düşük maliyetli pipeline kur.

Bu doküman, kod yazan AI'ın projeyi **ürün, veri, backend, frontend ve tasarım** olarak bütüncül şekilde anlaması için yeterli. Bir sonraki adım olarak bu brief'i parçalara bölüp önce **database schema + pipeline mimarisi**, sonra **Discover UI**, ardından **Radar/Studio** için ayrı prompt'lar hazırlayabiliriz.